User Guide for Generating Code Complexity Metrics

Visual Studio is equipped with a native tool that analyzes the code complexity of the entire project called Code Metrics Viewer (CVM). CVM takes five measurements:

- Maintainabilty Index
- Cyclomatic Complexity
- Depth of Inheritance
- Class Coupling
- Lines of Code

Running the Code Metrics Viewer

1. CMV is accessed by clicking on 'Analyze' in the Visual Studio toolbar.
2. Mouse over 'Calculate Code Metrics' which will pop open another panel out to the right.
3. Click on 'For Solution' to generate metrics for the entire solution (See Figure 5.1).
    - It is possible to generate the metrics of a single project.
    - To do this, go to the 'Solution Explorer' in the upper right pane.
        - If you do not see the 'Solution Explorer' pane, click on 'View' in the Visual Studio toolbar, and select 'Solution Explorer'.
    - In the 'Solution Explorer', single click to highlight the project you want to analyze.
    - This will add 'For <name of project>' underneath the 'For Solution' that pops up after following Steps 1-3.
    - As shown in Figure 5.1, one would be able to choose whether to analyze the entire Solution or only the CWMasterTeacher3 project.

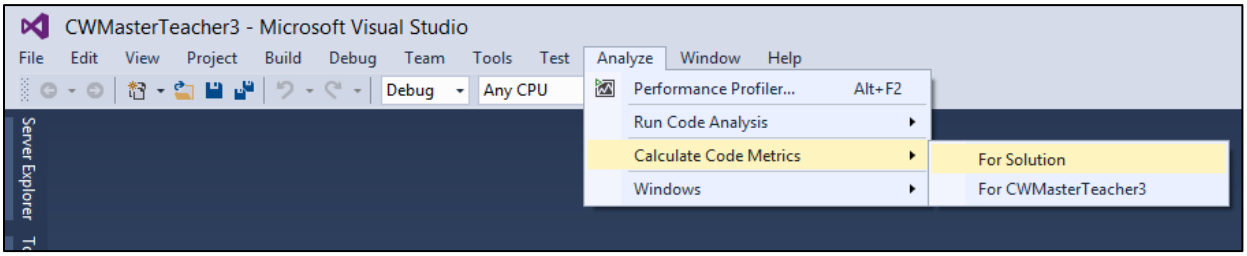

Figure 5.1 Visual Studio Code Metrics Tool

Code Metrics Viewer Results

After the CMV has completed its work, the results will be displayed in the larger lower pane of Visual Studio. Figure 5.2 shows what it looks like after the arrow next to CWMasterTeacher3 has been clicked to open the next level of the project's hierarchy. There are six columns total that are of interest:

1. Hierarchy
2. Maintainability Index
3. Cyclomatic Complexity
4. Depth of Inheritance
5. Class Coupling
6. Lines of Code

Columns 2-6 are the code metrics accompanied inside by numerical values that we will use to determine the complexity of the code.

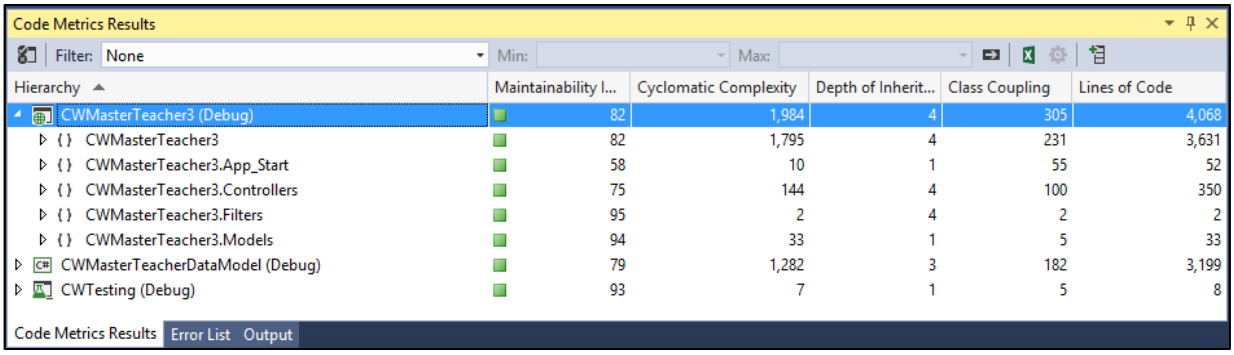

Figure 5.2 Visual Studio Code Metrics Tool Results

Figure 5.3 shows the expansion of the CWMasterTeacher3 project hierarchy to the member level of AdminChangePasswordViewModel.

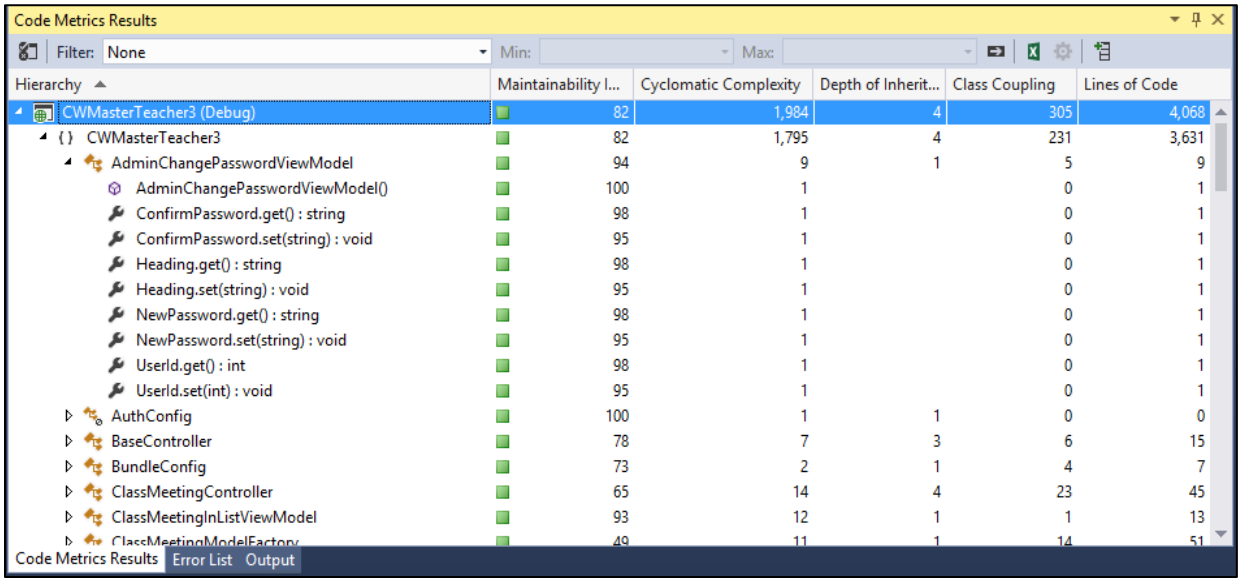

Figure 5.3 Code Metric Results at Subroutine Level

By right clicking on a class (signified by the brownish orange connected boxes icon) or member of a class (signified by the purple wireframe cube or wrench icons), options will appear (Figure 5.4), such as “Go to Source Code” that allows the viewing of the highlighted object’s source code in the main upper pane of Visual Studio.

In the same options (Figure 5.4), if ‘Open Selection in Microsoft Excel’ is selected, then Visual Studio will export all highlighted object’s metrics to a spreadsheet in Microsoft Excel.

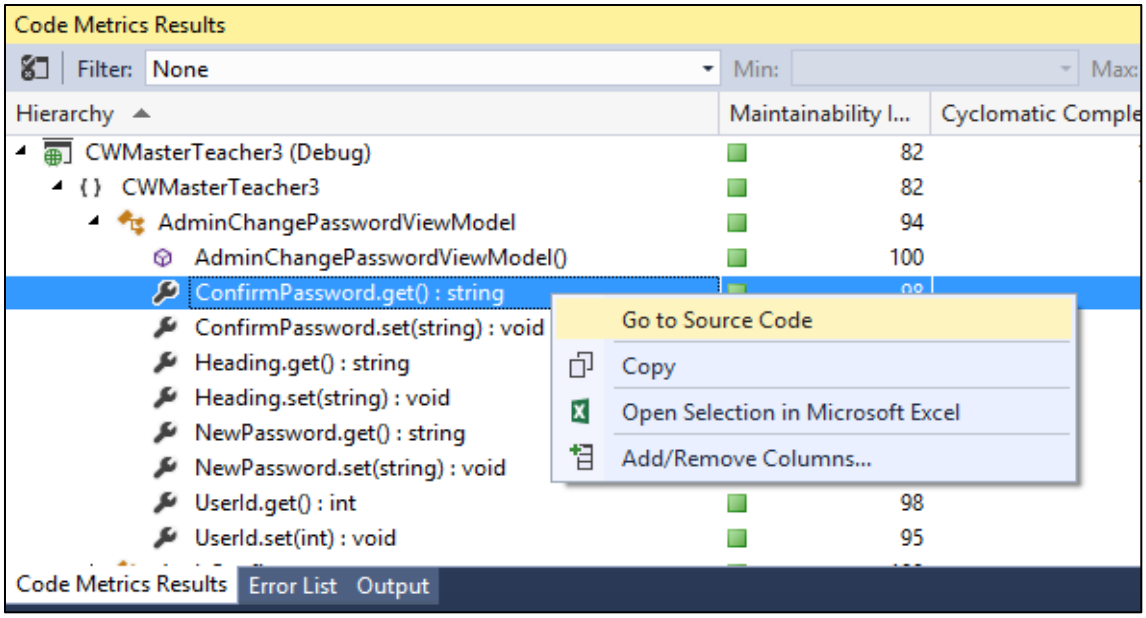

Figure 5.4 Code Metric Results Source Code Location

Filtering the Results

Next to 'Filter', in the menu bar of the Code Metric Results, is a drop down box that enables the results pane of CMV to be manipulated. The six default choices, 'None' and the five quantitative metrics, are left most oriented. The indented choices in Figure 5.5 are saved filters that have been created and previously used. This gives quick access to previous analysis without having to enter minimum and maximum values over and over.

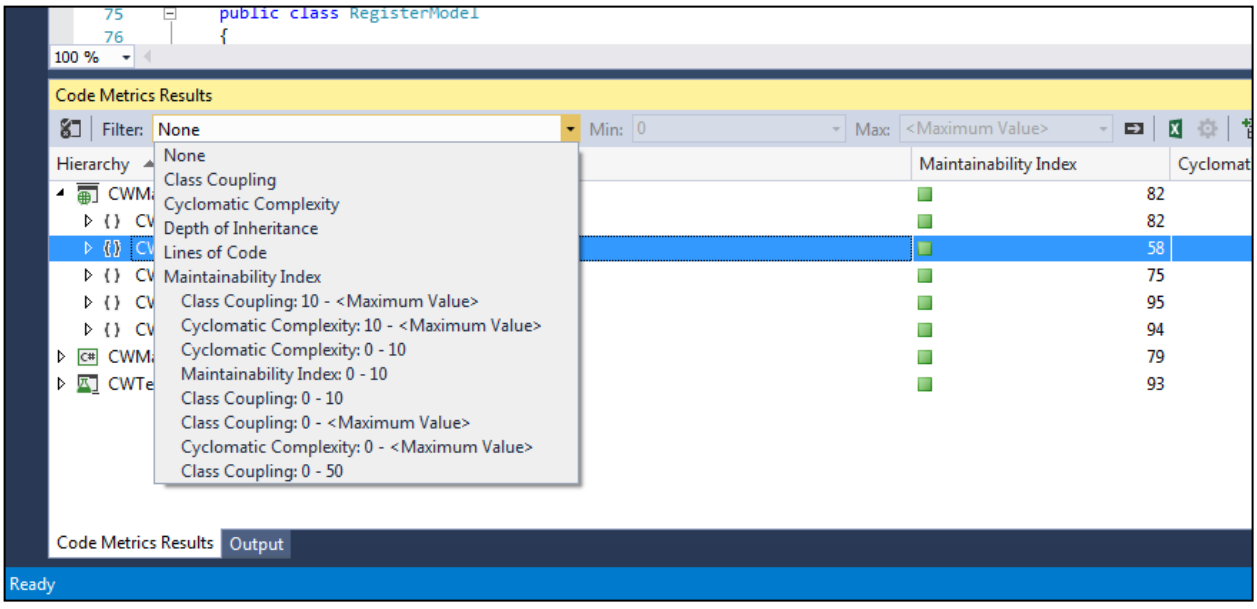

Figure 5.5 Applying Filter Drop Down Box

In order to create a filter that will also be saved for later use:

1. In the ‘Filter’ drop down box select one of the default left most quantitative metrics.
2. The ‘Min’ and ‘Max’ boxes will no longer be grayed out (Figure 5.6).
3. Enter numerical values into these boxes.
    a. These values determine the range of the filter to apply to the column selected for filtering.
    b. Figure 5.7 shows a filter selected for Cyclomatic Complexity with a Min of ‘5’ and a Max of ‘<Max value>’ (<Max Value> means no upper limit).
4. Click the right pointing arrow next to the ‘Max’ box to apply the filter.
5. Expand the hierarchy to determine the filters results
    a. **Note:** The filter applies to all levels of the solution.
6. Figure 5.7 shows an expanded CWMasterTeacher3.Controllers
    a. The expanded classes that show nothing in them contain members that do not go above the minimum threshold of ‘5’, therefore they are not displayed in the CMV Results after the filter is applied.

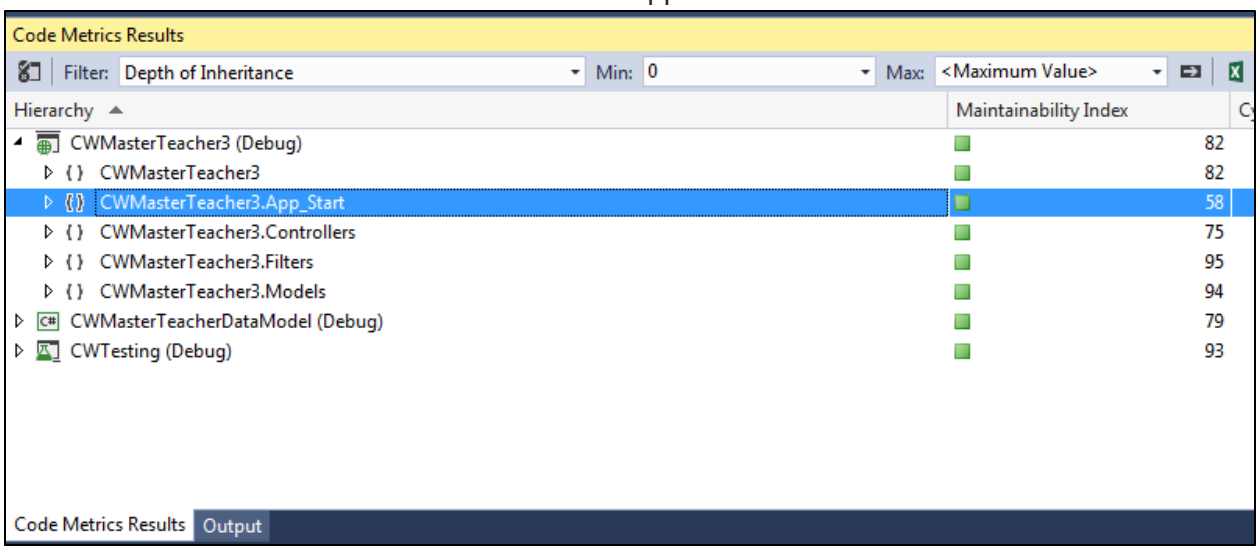

Figure 5.6 Minimum and Maximum Values for Filter

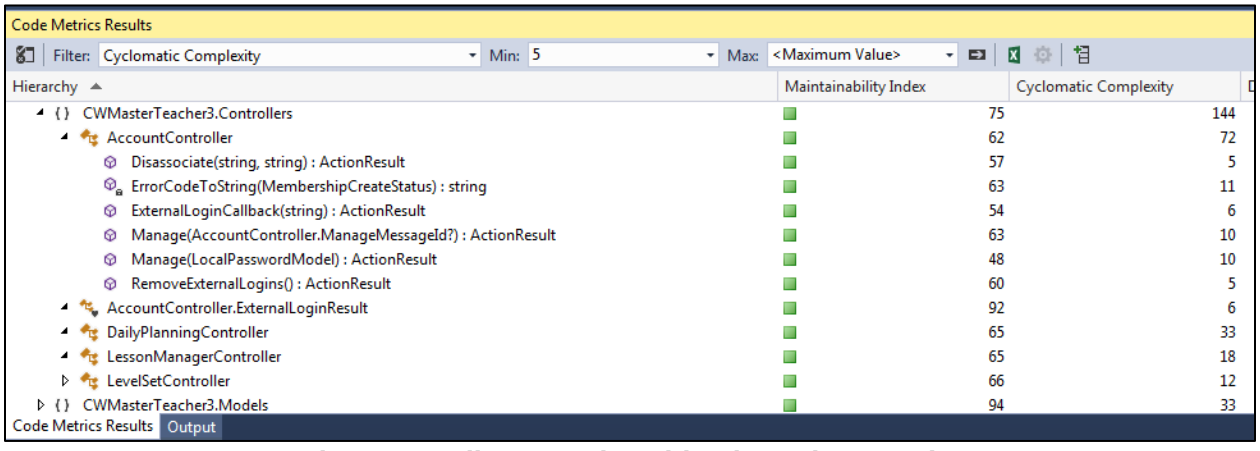

Figure 5.7 Filter Results with Min and Max Values

Exporting CMV Results to Microsoft Excel

Visual Studio can export the results of the Code Metrics Viewer to a Microsoft Excel Spreadsheet. From the Code Metrics Results pane it will export the results of your filtered search. If a filter was not applied, then it will export the entire solution from the top down to the member level. If a filter is applied, then it exports only what falls within the range of Min and Max values.

To export the CMV Results to Excel:

1. Apply or don't apply a filter as described in the 'Filtering the results' section.
2. Click the green box with a white 'X' in it next to the right pointing arrow 'Apply Filter' button as seen in Figures 5.5, 5.6, 5.7.
3. It might take a moment for Excel to come open and display the spreadsheet, but that is all it takes. It is very simple.

Detailed Description of Metrics

The following information provides a more detailed description of each metric mentioned in this document. This will provide a better understanding of each metric that will be used to measure code complexity.

Cyclomatic Complexity

Cyclomatic Complexity is the number of paths through a segment of code. Cyclomatic Complexity aids in defining the minimum number of test cases required for a module to have complete code coverage. It is used during the software development cycle to quantify maintainability and testability. The Cyclomatic Complexity of a program is calculated from a control flow diagram representation. Cyclomatic Complexity is calculated as follows:

$$\textit{Cyclomatic Complexity} = E - N + 2p$$

Where ***E*** is the number of edges of the graph, ***N*** is the number of nodes of the graph, and ***p*** is number of connected components. The following is the control flow diagram representation of a program:

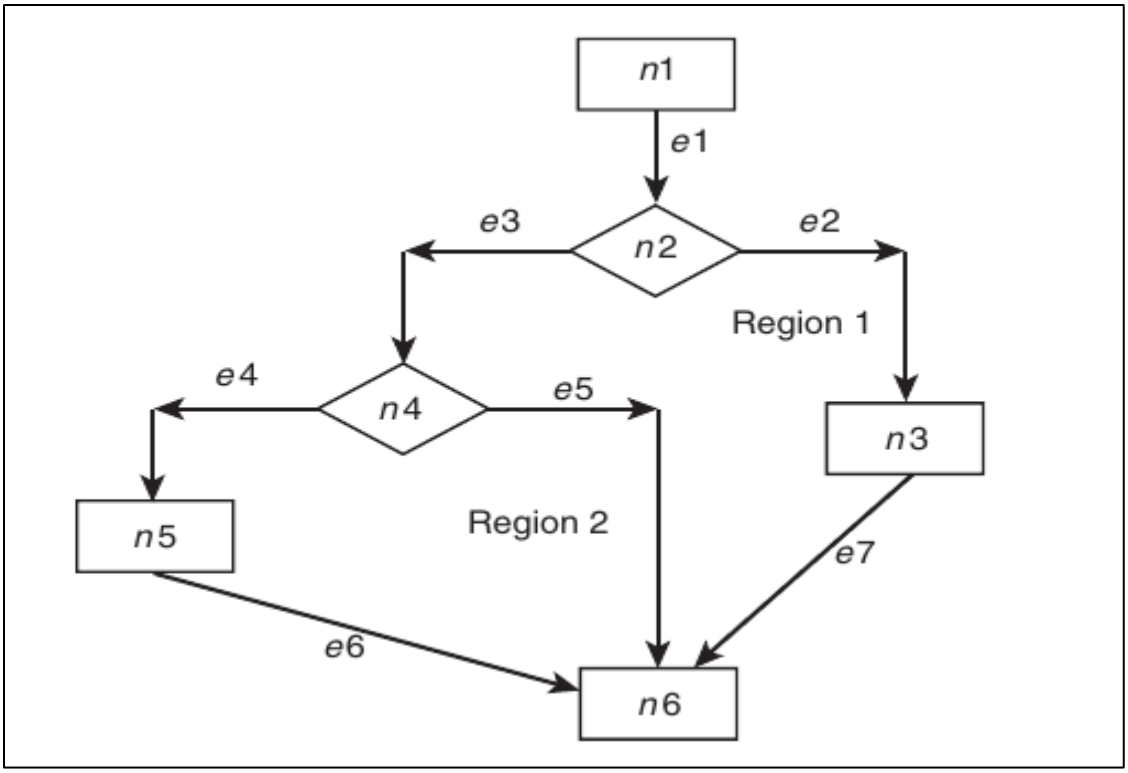

Figure 6.1 Control Flow Diagram

The calculation for the control flow diagram above is:

$$\textit{Cyclomatic Complexity} = E - N + 2p$$
$$= 7 - 6 + 2(1)$$

A program that has a complex control flow graph will require more tests to achieve good code coverage and will be less maintainable. It should be noted that having full code coverage does not mean that all of the behavior of the code is tested. More test cases are

often needed to ensure that the code segment behaves properly under different conditions. The number of tests required for the segment is often more than triple the Cyclomatic Complexity.

According to the Engineering Institute, a Cyclomatic Complexity number from 1 through 10 may be viewed as low risk and simple while a number greater than 50 is viewed as high risk or extreme.

Maintainability Index

The Maintainability Index is used to quickly identify trouble areas in the code. It is based on three metrics: Halstead Volume, Cyclomatic Complexity, and Lines of Code. The formula for calculating the Maintainability Index is as follows:

$$\textbf{\textit{Maintainability Index = MAX(0,(171 – 5.2 * ln(Halstead Volume) – 0.23 * (Cyclomatic Complexity) – 16.2 * ln(Lines of Code))*100 / 171)}}$$

Halstead Metrics measure a program module's complexity directly from source code, with emphasis on computational complexity. It measures complexity directly from the operators and operands used in the module. Halstead Volume is calculated as follows:

$$\textbf{\textit{Halstead Volume = N*log}}_2\textbf{\textit{*n}}$$

The ***n*** in the Halstead Volume represents $n_1 + n_2$ which stands for the total number of distinct operators + total number of distinct operands. Examples of a distinct operator are +,-,if, etc. The ***N*** in the Halstead Volume represents $N_1 + N_2$ which stands for the sum of all occurrences of n_1 + sum of all occurrences of n_2.

Cyclomatic Complexity measures the maximum number of independent paths through a segment of code as follows:

$$\textbf{\textit{Cyclomatic Complexity = E – N + 2p}}$$

Where ***E*** is the number of edges of the graph, ***N*** is the number of nodes of the graph, and ***p*** is number of connected components.

Higher values of the maintainability index mean better maintainability. A value between 20 to 100 indicates that the code has good maintainability, a value between 10 to 19 indicates that the code is moderately maintainable, and a value between 0 to 9 indicates that the code has low maintainability.

Source Lines of Code

The Source Lines of Code (SLOC) count is based on the number of Source Lines of code.. A high count might indicate that a type or method is trying to do too much work and some of the code should be pulled out into a helper method. It can also indicate that the class or method might be hard to maintain.

What constitutes a good method length is a common subject of debate. The only thing most sources agree upon is that shorter is better. Methods that are long, and require scrolling while being viewed, force developers to keep more of the code in mind. This increase in cognitive load makes the code segment more difficult to read and understand. For these requirements we have selected 30 SLOC as a good upper bound.

Class Coupling

Class Coupling measures how many classes a single class uses in its code through parameters, local variables, return types, method calls, generic or template instantiations. Each class that appears on the code is measured only once no matter how many times it is used throughout that class. Class Coupling has been proven to predict software failure as it is often aimed to have “loose” coupling. Coupling is determined by its different levels of coupling, from which it includes the following:

- Content: Internals are shared between two units
- Common: Same global data is used
- Control: Data being passed affects control
- Stamp: Passing more data than are needed
- Data coupling: Only the necessary data is being passed

Studies have shown that 9 is the number to be the most efficient upper limit for Class Coupling metrics, as a high coupling indicates a design that is difficult to reuse and maintain as it would contain interdependencies on other types.

Depth Inheritance Tree

Classes that inherit from other classes have the potential to reuse code from the class they directly extend and any classes that are extended from those classes.

As an example a dog type and a cat type could inherit common traits from an animal type. This reduces code duplication and builds a logical coupling between dog and cat to the base type animal. In this example dog and cat both would have a depth of inheritance of 2. A fourth type Labrador inheriting from type dog would have a depth of inheritance of 3. Labrador would have access to all of the traits of type dog and type animal.

Higher Depth Inheritance Tree (DIT) values indicate larger inheritance trees. The more classes a module inherits from the more complex the overall dependency graph will be. In addition, it’s more difficult to predict the actual behavior of the inheriting class.

It should be noted that while high DIT values indicate high design complexity it also indicates that there is a high potential for reusing existing code in the inheriting module. There’s little empirical evidence to support any specific value for DIT. For the requirements the value of 6 was chosen.